Summary of slides

Conceptual Modeling

What?

Capture ideas and concepts to reasoning; activities.

Why?

- Understan the real-world an the sistem as it
- Envision the sistem we want with Key decisions
- To specify a solucion guided by a plan made by communication

When

During the software developmen process whit different perspectives

- Analysis
- Design
- Specification
- Implementation
- Deployment

UML (Unified Modelin Langua-ge)

Case diagram Need a software
Class diagrams Describe classes of objects
Interaction diagrams how objects interact
State charts behavior of object

Organize software into Loosely coupled Layers

Layers provide abstraction to higher layers, using his own components or lower-layer abstraction dependicn on them

Escaneado con CamScanner

DD MM AA

Agregation over Inheritance

- To reuse is better Aggregation over Inheritance
- Programming to an interface reduces coupling, An interface is the public part of a clas
- Use meaning ful names

  classes - nouns (Pascal Casing)
  variables - reflecting purpose (LowerCamer Casing
  metods - verbs

To improve code readability and maintainability

- Unit testing verify code, find errors and enable regression. Testing frameworks must be used

Introduction to the Unified Modeling Language

Is an expressive and customizable language that supports incremental development

60's Initial OO ideas

70's More Programming languages

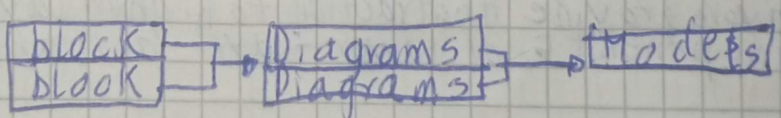
80's OOPLs gain widespread use

90's Method Wars and standarization, Patterns

Current Full-Life-Cycle

Building Blocks

Scoping
Structural
Behavioral
Organizational
Annotational
Extension



Escaneado con CamScanner

Elements

Definition and scope Reach of sistem

Structural Static part

Extension Extending UML

Behavioral Things dynamics

Organizational Things Grouped

Annotation Notes and clarifications

Diagrams Visual views

Views Describe a sistem

Use-case view
Logical View
Implementation view
Process view
Deployment view

Valve and understanding depend on process an perspective

Analisis Models the real world (Understanding)

Specification Defines the system (Planification)

Implementation Software and concepts (Construction)

Primary activities A software needs requirements gatering, design, etc

Cross-cutting Manage people and resources

Types of Diagrans

Structural

Class: classes interfaces and relationships (static design view

Objecti: objects and relationships (photo of a moment

Component Software and dependencies (implementation view

Norma

Escaneado con CamScanner

Deployment physical nodes (hardware) and components

Behavioral

Use Case cases and actors (interact with the system)

Sequence Cronological order of messages

Collaboration structural organization

State Chart states, transitions and events

Activity Show and chontrol of the flow

More on Views collection of diagrams that describe a system from a specific perspective

Select the correct views for your project and change control

Abstraction Different level of details separating or changing detail on models

Complex group elements with notes or colors to identify patterns

Hint and Tips

Diagrams need clear connections and names without simplify so much

Escaneado con CamScanner

Use Cases

Basic Concepts

Actors External entities to the sytem that interacts with it (users, other systems, hardware) They have goals and are represented by stick figures Not all actors are users

Use Cases Are what the user needs to archive even if it implementation is different, capture the visible functionality of the sistem, and are named based in the users goal and don't take as a important the desing decisions

Use Cases ~~Address~~ the What and Why (the goal)

System interacton Address the How (steps)

Use Case Diagrams

Composed by Actors, Use Cases, Relationships, Subjects

Relation with actors Generalization / Specialization

Relation with Use Cases:

<<Extend>> A variaton or addi of a base use case This one new exist on its own

<<Include>> This requires yes or yes the functionality of another, its like aggregation

Generalization A use case is a subtype of another

Good design Communicating a specific aspect of the systems only the essentials and a level of abstraction

Hints & Tips

Identify Actors Ask who uses the core functionality who provides or consumes inputs/outputs, requires support and what systems interact with the solutions

Norma

Escaneado con CamScanner

DD MM AA

Modeling Actors An actor represent a role a single person can hold multiple roles, an actor several cases dont think on exeessive detail

Identify Use Cases List actors and their activities, and search for external events that system respond. Be patient and dont overcrowd the diagram

Drawing the Diagram Give a name avoid crossing lines group related and use notes and colors

Benefits

functions are visible for user
Help to identify a specific functionality
Document opportunities for reuse
Fundation for planning, development and testing

Escaneado con CamScanner

Objects, Classes and Basic Class Diagram

Objects

Objects are the fundamental building blocks of any object-oriented system

- Represents any thing in the domain
- Has a boundary in space and/or time
- Is an abstraction
- Encapsulates state and/or behavior
- Has its own identity (two objects can have the same values and still be distinct).

What is usually not modeled as an object:

- Event (sometimes yes/no)
- Relationships between objects (most of time)
- Behavior and constraints (almost never)

Classes

Classes are abstractions that allow us to deal with collections of objects that share common characteristics

Human naturally classify similar things by their observable properties; classes formalize this idea.

Typical examples in a Student Records Management System:

Student, Course, Semester, College, Major

Rules

- Should be a noun

Norma

Escaneado con CamScanner

- Singular
- Meaningful from the problem-domain perspective
- Upper Camel Case
- package path

Three Perspectives

~~ML~~ UML elementes are interpreted differently

Perspective	Class	Attribute	Operation
Analysis	set of objects in the problem domain	Relationship to another object	Ways in which objects interact
Specification	Interface that encapsulates software representation of objects	Obligation to provide or manage a certain piece of information	Obligation to provide a service
Implementation	Abstract data type/ code class	Data member (Java C++ C#)	Method or member function

Class and Object Syntax

Class a box divided into compartments

Object Very similar syntax, but:

- Name is underlined
- The form instanceName:ClassName.
- The instance name may be blank
- Attributes can show values

Attribute syntax

[visibility] name[multiplicity] [: type][=initial-value] [{property-string}]

Visibility: + public, # protected, - private

Escaneado con CamScanner

DD MM AA

Operation syntax

[visibility] name([parameter-list]) [: return-type] [{property-string}]

The name is usually a verb in Lower Camel Case

5. Relationships in Class Diagrams

According to the UML meta-model, relationships are organized as follows

- Relation
  - Generalization
  - Association -> Binary Association / N-ary Association -> Aggregation -> Composition
  - Dependency

Generalization

Represents the "is-a" relationship. From an analysis perspective is a pure generalization/specialization. From specification and implementation perspectives it can represent interface inheritance subtyping or implementation

Associations

Structural relationships that specify objects may be connected to objects of another class

- May be named
- Navigation can be uni-directional bi-directional
- Bi means more than just traverse the association

Agregation and Composition

Aggregation (hollow diamond): a weaker "is-part-of" can have independent lifecycles

Composition (filled diamond): a strong form of aggregation the lifecycle is tied to the whole one

Norma

Escaneado con CamScanner

Multiplicity

Defines how many objects of a clas can be linked from a given association, a single number a * (many) a range min, max or a sequence

Dependencies

A "uses" relationship represented by a dashed arrow can change in the target element may required a change in the source element

Can be defined among classes, packages, etc
common types are <<become>> and <<instance of>>

Characteristics and Best Practices of Class Diagrams

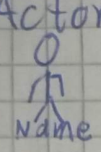
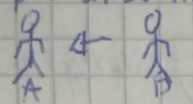
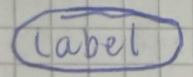
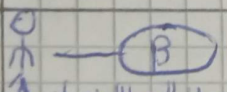
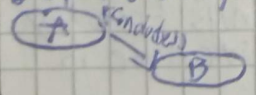
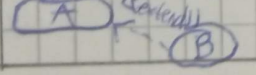
- Describe the types of objects, their properties and relationships
- Similar to ER models, but show classes not data entities
- Include objects, packages, constraints and annotations
- Focus on one topic and keep a level of abstraction
- All diagrams together from the system's "model"

Common pitfalls

- Use only class diagrams
- Mixing perspectives
- Wrong or mixed levels of abstraction
- Too many dependencies or overly large diagrams

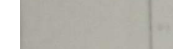
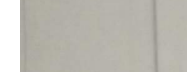
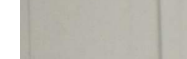
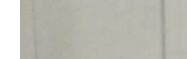
Escaneado con CamScanner

Use Case Diagrams

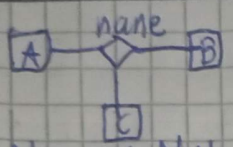
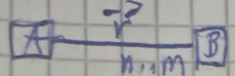
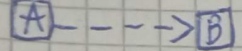
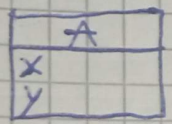
Component	Meaning	Best Practices
Actor 	Someone/something outside the system with a goal	Nouns, capitalize each word
Generalization/ Specialization 	"B is an "A" -> set of B is a subset of A	Overlapping lines on the triangle is acceptable when there are many
Use Case 	A goal an actor with respe to the system	Clear, concise label that is understandable out of context.
 Actor "has" Use Case	Actor A has the goal of Use Case B.	No arrowheads or adornments, solid line only.
Include Dependency 	Goal A includes sub-goal B. Archieving A requires also archieving B	Break high-level goals into sub-goals, but stop before describing how the system works
Extended Dependency 	Goal B adds to or alters goal A under special/exeptional conditions.	Arrow points from the extension (B) to the base goal (A). Use only for atypical cases

Note Note	Free-text comment connected by a dotted line	Use while brainstorming, later move content into formal elements.

Class Diagrams

Class Name	Set of objects related	Noun; Capitalize each word
Generalization Specialization 	"B is an A" → Every B is also an A	Overlapping lines on triangle when there are many
Named Binary Association 	Persisten links between A and B with meaning "A rB"	Use precise verb phrase
Aggregation 	"B is a subpart of A" / "A contains B"	For part-whole relationships
Compositions 	Strong aggregation: if A is destroyed its parts B are also destroyed	For a part who belongs to only one whole and its lifetime is tied to it

Escaneado con CamScanner

 Named N-Nary Association	Persistent links among three or more classes	Name should be a complete sentence including all class names
Multiplicity constraint 	How many objects of one class relate to one of another	Double-check, Place only on associations
Dependency 	A may depend on B (not persistent)	Use sparingly in analysis
Attributes 	Properties or placeholders of the class	Prefer association over early attributes in analysis

Escaneado con CamScanner